Relational Databases



DRIM

Relational Databases

Relational Database

A relational database is a type of database management system (DBMS) that organizes data into tables consisting of **rows** and **columns**. It is based on the relational model, which was developed by Edgar F. Codd in the 1970s.

In a relational database, data is structured in a way that allows relationships to be established between tables using common attributes.

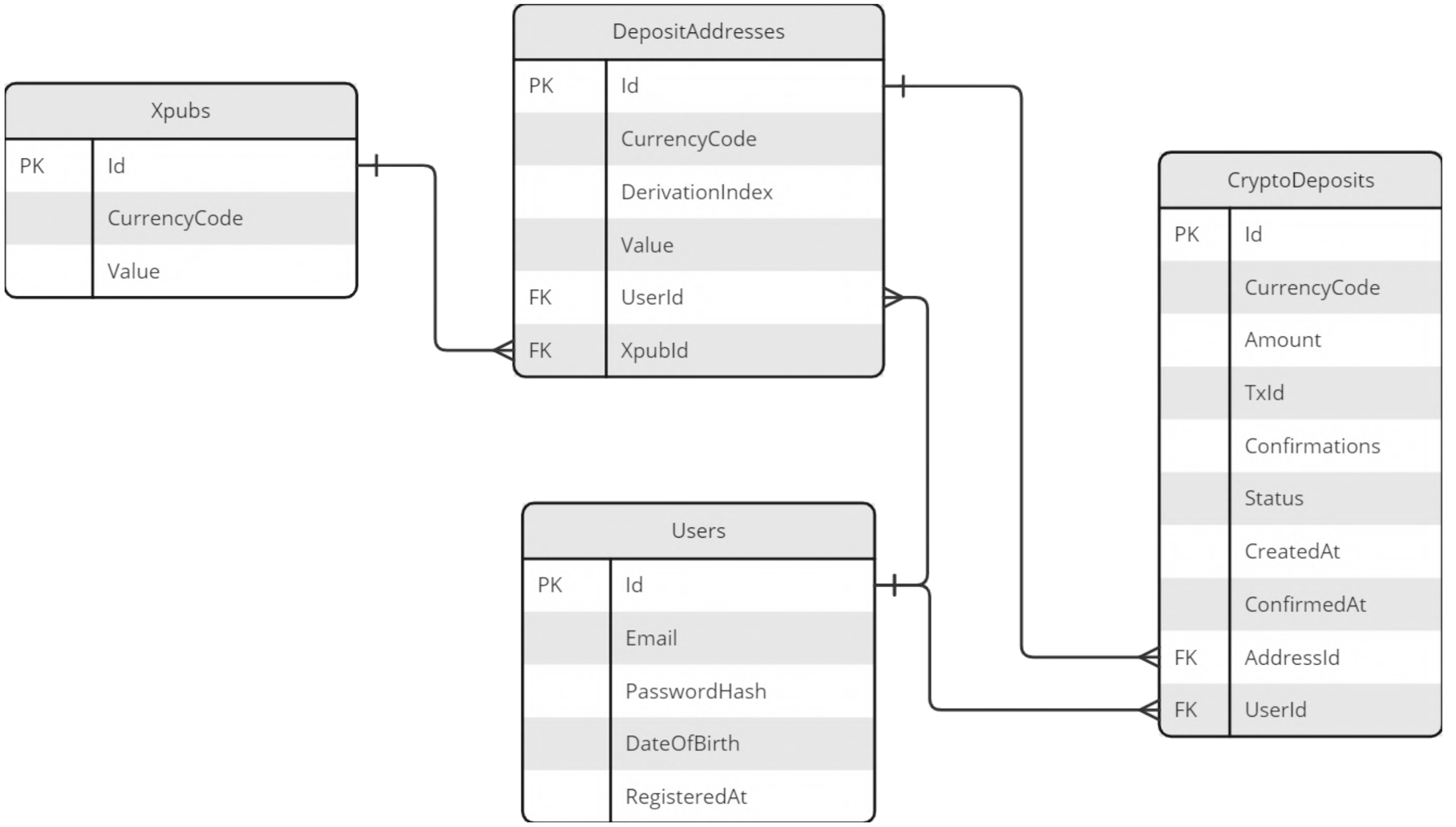
DRIM

Table

Id [PK] integer	Email character varying (64)	PasswordHash character varying (256)	DateOfBirth timestamp without time zone	RegisteredAt timestamp without time zone
1	sam@cryptobank.com	hash1	1995-01-30 00:00:00	2023-06-29 00:00:00
2	tom@cryptobank.com	hash2	1997-02-17 00:00:00	2023-06-30 00:00:00
3	jack@cryptobank.com	hash3	2001-06-05 00:00:00	2023-07-01 00:00:00
4	john@gmail.com	hash4	2000-06-05 00:00:00	2023-07-02 00:00:00

DRIM

Relations



Other Database Types

- Key–Value Databases
- Document Databases
- Hierarchical Databases
- Network Databases
- Columnar Databases
- Graph Databases
- Time–Series Databases
- Spatial Databases

SQL

SQL stands for **Structured Query Language**. It is a programming language specifically designed for managing and manipulating relational databases. SQL allows users to define, retrieve, modify, and manipulate data stored in a database.

SQL provides a standardized syntax and set of commands for performing various operations on relational databases, such as creating and modifying database structures (tables, views, indexes), inserting, updating, and deleting data, and querying the database to retrieve specific information.

DDL vs DML

- **Data Definition Language (DDL)** includes commands for defining and managing the database structure, such as creating tables, modifying table structures, and defining relationships between tables

- **Data Manipulation Language (DML)** includes commands for manipulating data in the database, such as inserting new records into tables, updating existing records, and deleting records

Primary Key

A primary key is a unique identifier for a particular record or row in a table. It is used to uniquely identify each row in a table and ensure that no two rows have the same identifier. Properties of primary keys are:

- Uniqueness
- Non-null
- Single value
- Indexing
- Referential integrity

Foreign Key

A foreign key is a relational database concept that establishes a relationship between two tables. It is a column or a set of columns in one table that refers to the primary key in another table. The purpose of a foreign key is to enforce referential integrity, ensuring that the data in the related tables remains consistent.

Constraints

A table constraint refers to a rule or condition that is applied to one or more columns in a table to enforce data integrity. Table constraints define the valid values or conditions that must be satisfied by the data stored in the table.

- Primary Key Constraint
- Foreign Key Constraint
- Unique Constraint
- Check Constraint
- Not Null Constraint

Joins

In SQL, a join is used to combine rows from two or more tables based on a related column between them. It allows you to retrieve data from multiple tables by specifying the relationships between them. There are different types of joins available in SQL, including:

- Inner Join
- Left Join (or Left Outer Join)
- Right Join (or Right Outer Join)
- Full Join (or Full Outer Join)
- Cross Join (or Cartesian Join)

Transactions

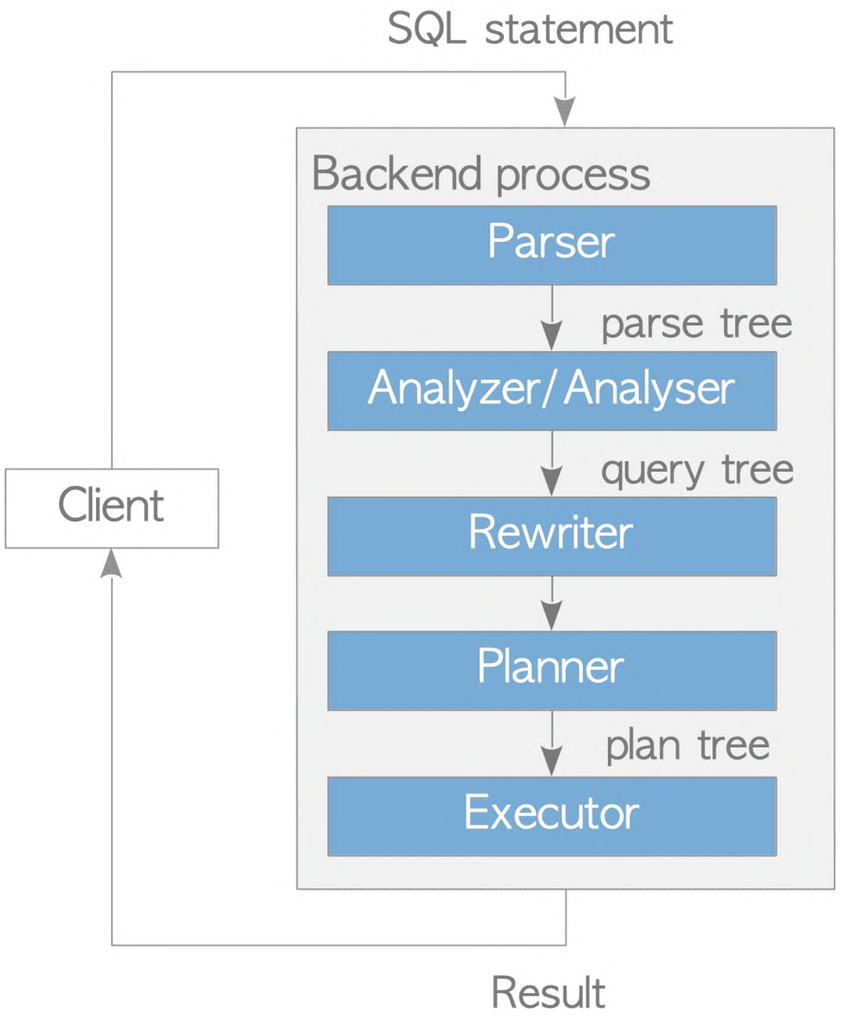
A database transaction is a unit of work that is performed on a database. It represents a logical and atomic operation that either completes in its entirety or is rolled back if any part of it fails. Transactions ensure data consistency and integrity in a database by allowing multiple database operations to be treated as a single, indivisible operation.

In a typical database transaction, the following properties are maintained, commonly known as the ACID properties:

- Atomicity
- Consistency
- Isolation
- Durability

DRIM

Postgres Architecture



Execution Plan

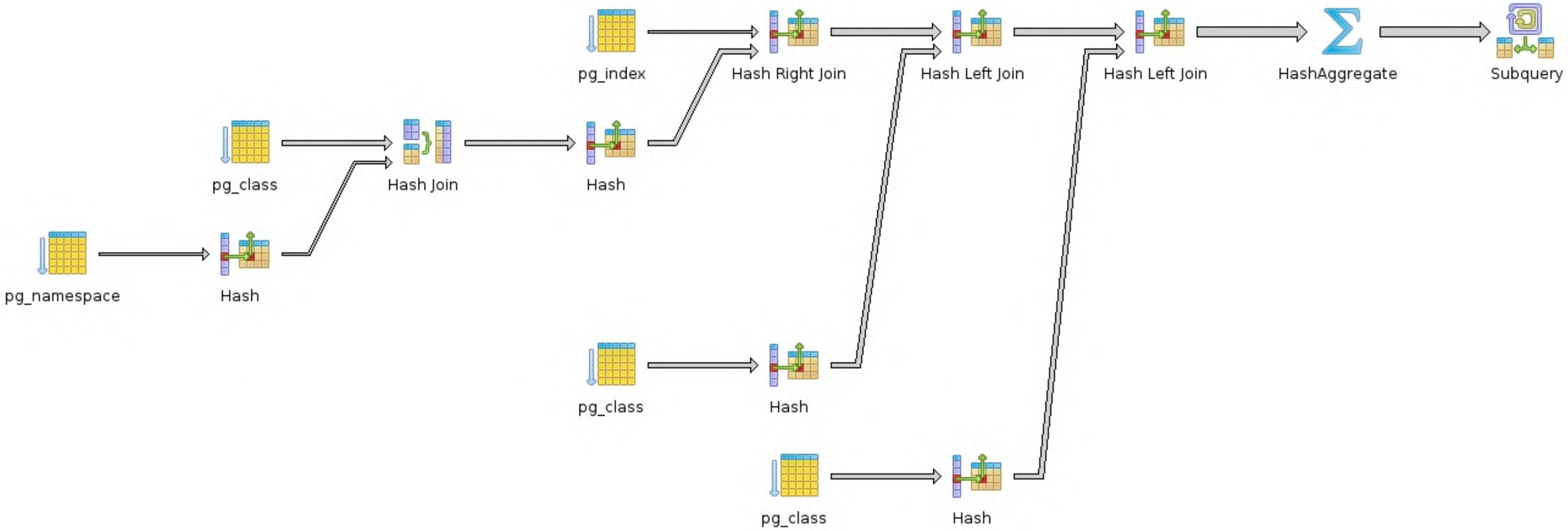
An execution plan, also known as query plan or query execution plan, is a detailed blueprint or strategy created by a database management system (DBMS) to efficiently execute a database query. It is a set of instructions that the DBMS uses to access and manipulate data in order to produce the desired query result.

When you submit a query to a DBMS, the system analyzes the query and generates an execution plan. This plan outlines the steps the DBMS will take to process the query, including the order in which tables will be accessed, the type of operations to perform (such as joins, filters, sorts, etc.), and the specific algorithms and data access methods to use.

The execution plan is crucial for optimizing query performance. It allows the DBMS to choose the most efficient way to execute the query by considering factors such as available indexes, statistics about the data, and the cost of different execution strategies. The goal is to minimize the resources and time required to process the query while returning accurate results.

DRIM

Execution Plan



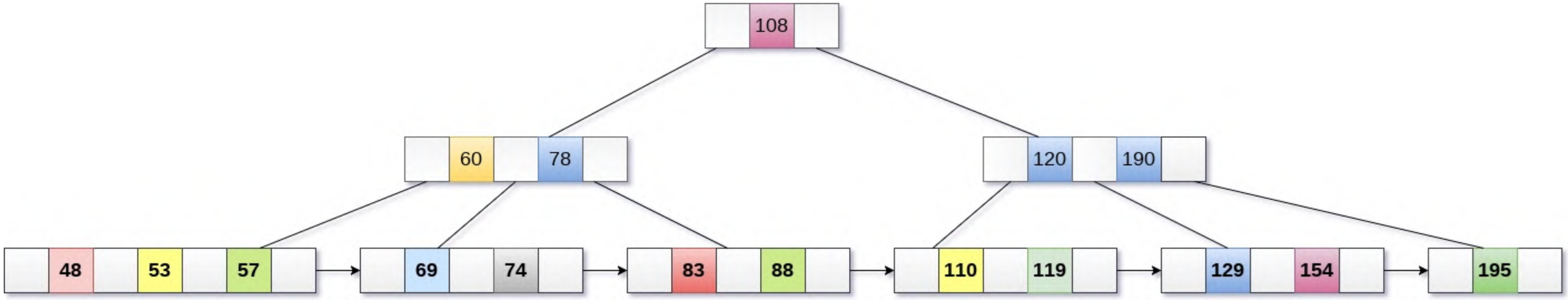
B+–Tree

B+ tree is a balanced tree data structure that is commonly used in computer science and database systems for efficient storage and retrieval of data. It is an extension of the B–tree data structure with additional properties that make it suitable for use in secondary storage devices like hard disks.

In a B+ tree, the data is organized in a hierarchical structure with multiple levels of nodes. The tree starts with a root node, which can have multiple child nodes. Each node in the tree, except for the root, has a fixed range of keys and a corresponding set of values or pointers. The keys in a node are sorted in ascending order.

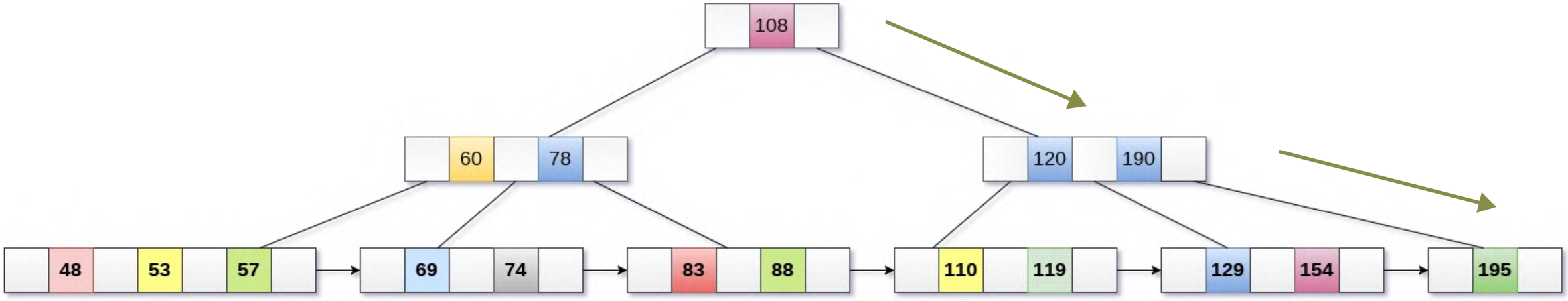
B+ trees are commonly used in database systems to index data on disk. The balanced nature of the tree ensures that search, insertion, and deletion operations have a logarithmic time complexity, making them efficient even for large datasets. Additionally, the use of disk blocks allows for efficient sequential access, making B+ trees ideal for range queries and scans.

B+-Tree

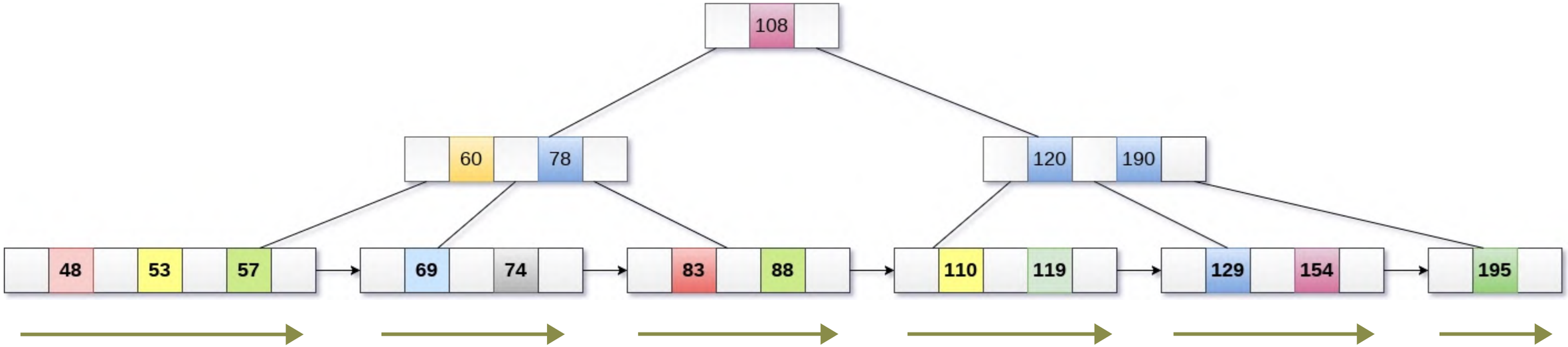


DRIM

Index Scan



Sequential Scan



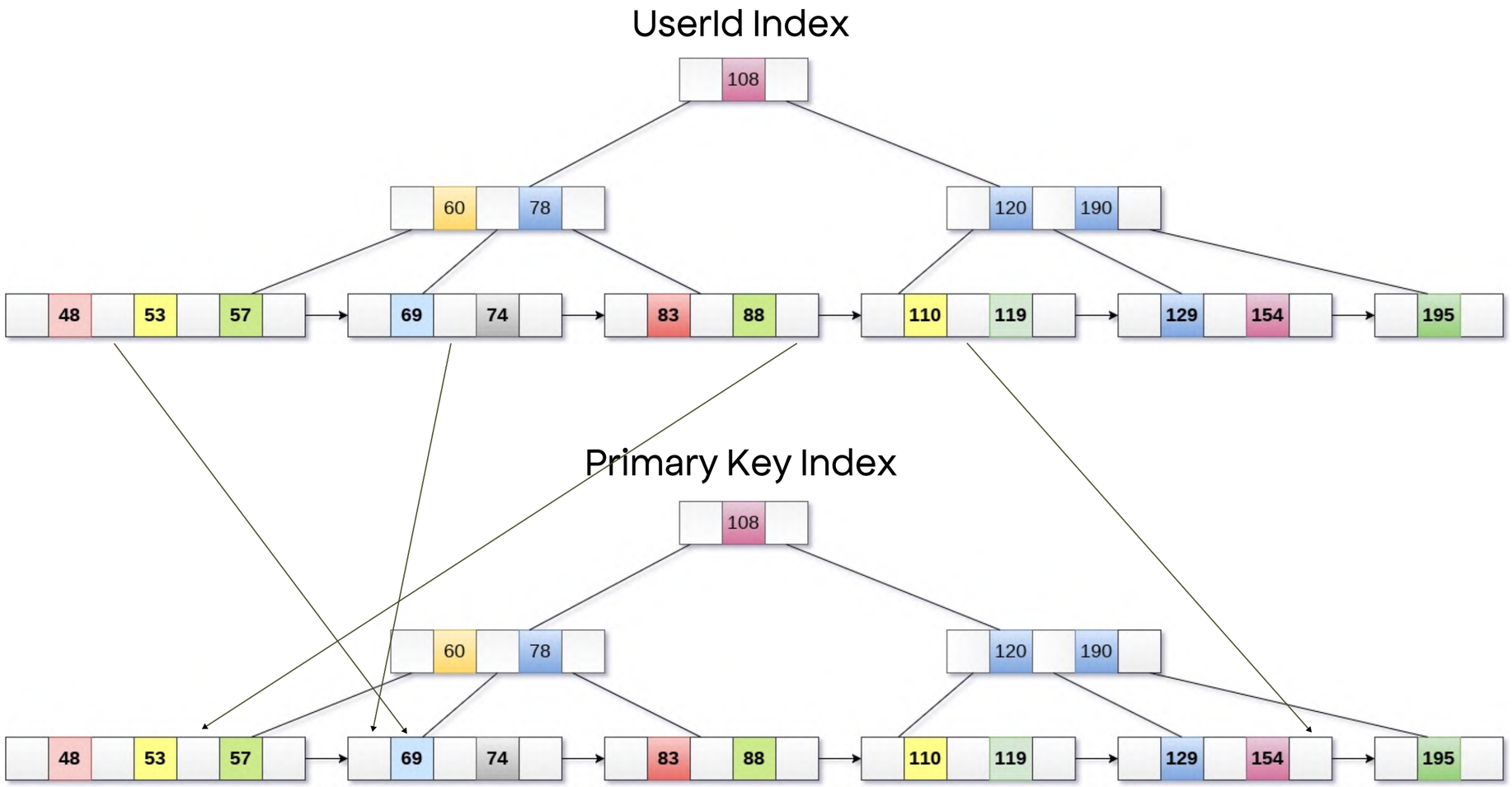
Indexing

Database indexing is a technique used to improve the performance of database systems by optimizing data retrieval operations. It involves creating additional data structures, known as indexes, that provide efficient access paths to the data stored in a database.

An index is essentially a copy of selected columns or fields from a table, along with a reference to the location of the corresponding rows in the table. This copy is organized in a specific data structure, such as a B-tree or a hash table, which enables quick searching and retrieval of data based on the indexed columns.

The primary purpose of indexing is to reduce the number of disk I/O operations required to satisfy a query. Without an index, the database management system (DBMS) would need to scan the entire table to locate the desired rows. With an index, the DBMS can use the index data structure to locate the relevant rows directly, leading to significant performance improvements, especially for large tables.

Indexing


UserId Index
108
60
78
120
190
48
53
57
69
74
83
88
110
119
129
154
195
Primary Key Index
108
60
78
120
190
48
53
57
69
74
83
88
110
119
129
154
195

Indexing

Pros:

- Faster Search
- Efficient Sorting
- Enhanced Concurrency
- Data Integrity

Cons:

- Increased Storage Space
- Overhead in Data Modification
- Maintenance Overhead
- Increased Complexity

DRIM

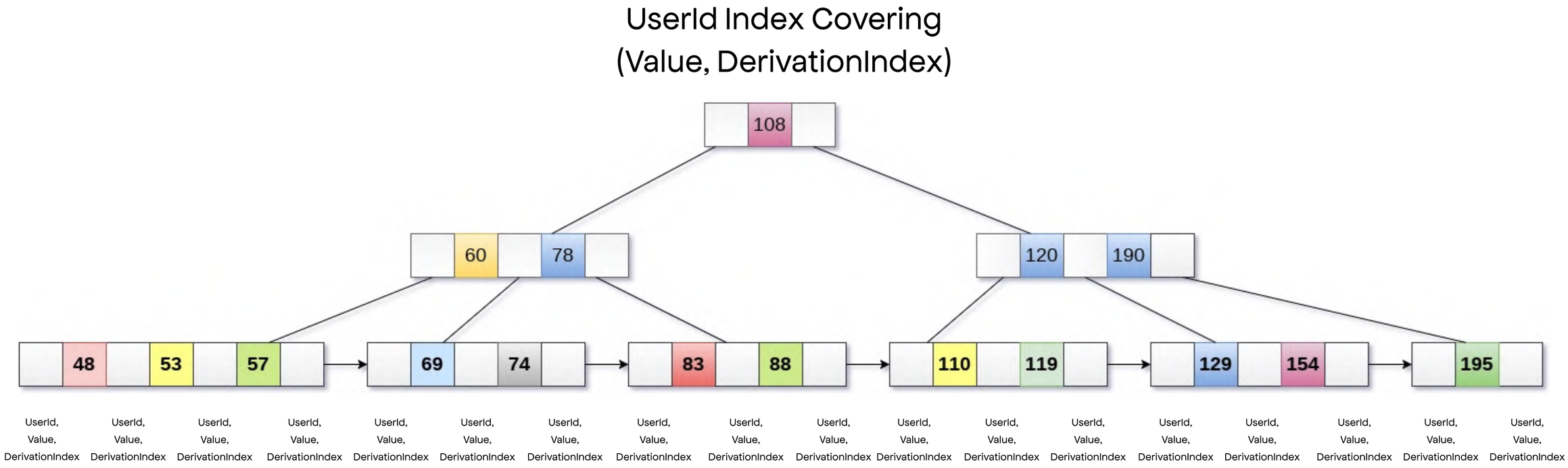
Covering Index

A covering index, also known as a non-clustered index, is a type of database index that includes all the columns required to fulfill a specific query without having to refer back to the actual data pages. In other words, it covers the query and provides a direct path to the data.

When a database query is executed, the database engine uses indexes to quickly locate the relevant data. Typically, an index contains a subset of columns from a table and provides a quick way to search and retrieve data based on those columns. However, if a query requires additional columns that are not included in the index, the database engine needs to perform additional I/O operations to retrieve the required data from the actual data pages, which can impact performance.

DRIM

Covering Index



Indexing Data Structures

- B+–trees
- Hash Indexes
- R–trees
- Bitmap Indexes

DRIM

Thank you! Questions?

